Sintaxis

Juan Manuel Rabasedas



Sintaxis

La sintaxis es la forma de un lenguaje.

- Sintaxis concreta: Secuencia exacta de caracteres que son programas sintácticamente válidos.
  $$1 + 2 + 3 \not\equiv (1 + 2) + 3 \not\equiv (1 + 2) \\ +3$$
- Sintaxis abstracta: La estructura esencial de los programas sintácticamente válidos.
  $$1 + 2 + 3 = (1 + 2) + 3 = (1 + 2) \\ +3$$

Lenguajes Formales

- Un lenguaje es un conjunto de palabras.
- Una palabra (cadena) es una secuencia finita de símbolos.
- Con ε denotamos la palabra vacía, o sea la secuencia de 0 símbolos.
- Un símbolo es un elemento de un conjunto finito denominado alfabeto Σ.
- Ejemplo:

$$
\begin{array}{rl}
alfabeto & \Sigma = \{a, b\} \\
palabras & \varepsilon, a, b, aa, ab, bb, \\
 & aaa, aab, abb, baa, bab, \ldots \\
lenguajes & \emptyset, \{\varepsilon\}, \{a\}, \{b\}, \{a, aa\}, \\
 & \{\varepsilon, a, aa, aaa\}, \\
 & \{a^n \mid n \geqslant 0\}, \\
 & \{a^n b^n \mid n \geqslant 0, n\ par\}
\end{array}
$$

Palabras de un Alfabeto

Dado un alfabeto Σ, se define el conjunto Σ^* como el conjunto de secuencias sobre Σ.
Inductivamente:

- $\varepsilon \in \Sigma^*$
- Si símbolo $s \in \Sigma$ y palabra $w \in \Sigma^*$ entonces $sw \in \Sigma^*$

Por ejemplo dado $\Sigma = \{0, 1\}$
$\Sigma^* = \{\varepsilon, 0, 1, 00, 10, 11, 000, \ldots\}$

Un alfabeto Σ finito genera infinitas palabras Σ^*

Lenguajes Formales

Dado un alfabeto Σ, un lenguaje L es

$$L \subseteq \Sigma^*$$

o equivalente

$$L \in \mathcal{P}(\Sigma^*)$$

Son ejemplos de lenguajes formales:
El conjunto de todos los programas sintácticamente válidos en un determinado lenguaje sobre un alfabeto.

- El conjunto de programas válidos de C es un lenguajes sobre el conjunto de caracteres ASCII.

Gramáticas Libres de Contexto

Establecen un conjunto de reglas que la escritura del lenguaje debe seguir.

- Las Gramáticas Libres de Contexto (**CFG**) describen lenguajes libres de contexto.
- Permiten la expresividad que caracteriza a la mayoría de los lenguajes de programación:
  - estructura anidada,
  - paréntesis balanceados,
  - definición de bloques.
- Puede ser reconocidos por un **autómata de pila**.

Gramáticas Libres de Contexto

Una **GLC** es una tupla (N, T, P, S)

- N es un conjunto finito de no terminales.
- T es un conjunto finito de terminales, donde $N \cap T = \emptyset$.
- S es el símbolo inicial, $S \in N$.
- P es un conjunto de producciones de la forma $A \to \alpha$, donde $A \in N$ y $\alpha \in (N \cup T)^*$.

Ejemplo: $G = (\{S, A\}, \{a, b\}, P, S)$ siendo P

$$
\begin{array}{lcl}
S & \to & \varepsilon \\
S & \to & aA \\
A & \to & bS
\end{array}
$$

Las producciones con el mismo lado izquierdo se pueden agrupar (BNF)

$$S \to \varepsilon \mid aA$$

También podemos escribirlas reemplazando $\to$ *por* $::=$

La relación de derivación directa

Para obtener el lenguaje generado por una gramática $G = (N, T, P, S)$ definimos una relación binaria $\Rightarrow$ para cadenas de $N \cup T$ como la menor relación tal que:

$$\alpha A \gamma \Rightarrow \alpha \beta \gamma$$

Cuando $A \to \beta$ es una producción de G
Para la gramática:

$$\begin{array}{lcl} S & \to & \varepsilon \mid aA \\ A & \to & bS \end{array}$$

Podemos tener las siguientes relaciones directas:

$S \Rightarrow \varepsilon$
$S \Rightarrow aA$
$aA \Rightarrow abS$
$SaAaa \Rightarrow SabSaa$

La relación de derivación

La relación de derivación $\Rightarrow^*$ es la **clausura reflexiva transitiva** de $\Rightarrow$.
Es la **menor relación** sobre cadenas en $N \cup T$ tal que:

- $\alpha \Rightarrow^* \alpha$
- $\alpha \Rightarrow^* \beta$ si $\alpha \Rightarrow \beta$
- $\alpha \Rightarrow^* \beta$ si $\alpha \Rightarrow^* \gamma \wedge \gamma \Rightarrow^* \beta$

Para la gramática G:

$$\begin{array}{lcl} S & \rightarrow & \varepsilon \mid aA \\ A & \rightarrow & bS \end{array}$$

tenemos que:

$S \Rightarrow^* \varepsilon$
$S \Rightarrow^* aA$
$aA \Rightarrow^* abS$
$S \Rightarrow^* abS$
$S \Rightarrow^* ababS$
$S \Rightarrow^* abab$

Lenguaje generado por una gramática

El lenguaje generado por una gramática $G = (N, T, P, S)$, se denota $L(G)$, y se define como:

$$L(G) = \{w \mid w \in T^* \wedge S \Rightarrow^* w\}$$

Un lenguaje L es libre de contexto (LLC) sii $L = L(G)$ para alguna GLC G .

Una cadena $\alpha \in (N \cup T)^*$ es una **forma sentencial** *sii $S \Rightarrow^* \alpha$*

Para la gramática G:

$$\begin{array}{lcl} S & \rightarrow & \varepsilon \mid aA \\ A & \rightarrow & bS \end{array}$$

$$L(G) = \{(ab)^i \mid i \geqslant 0\} = \{\varepsilon, ab, abab, ababab, \ldots\}$$

Árbol de Parseo

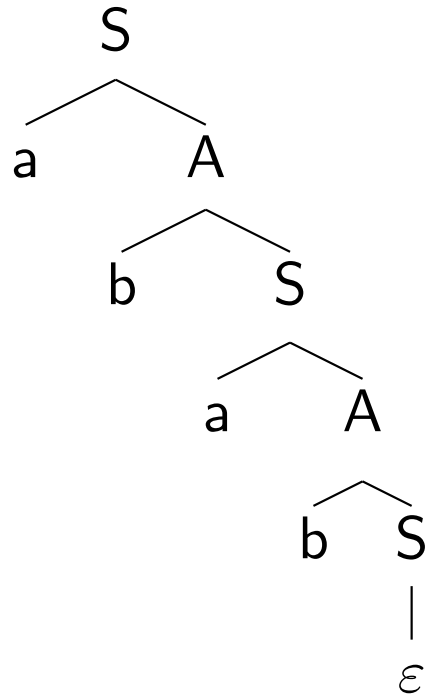
Un árbol es una derivación o árbol de parseo de un GLC $G = (N, T, P, S)$ si:

- cada nodo tiene una etiqueta en $N \cup T \cup \{\varepsilon\}$,
- la etiqueta de la raíz es S,
- la etiquetas de los nodos interiores están en N,
- si el nodo n tiene etiqueta A e hijos $n_1, n_2, ..., n_k$ (de izq. a der.) con etiquetas $X_1, X_2, ..., X_k$, entonces $A \to X_1, X_2, ..., X_k$ es una producción en P,
- si un nodo n tiene etiqueta ε, entonces n es una hoja y es hijo único.

Árbol de parseo: Ejemplo

Dada la gramática G:

$$\begin{aligned} S &\rightarrow \varepsilon \mid aA \\ A &\rightarrow bS \end{aligned}$$



La cadena $abab \in L(G)$ tiene el árbol de parseo

- La cadena que se forma con las etiquetas en las hojas de izq. a der. es el **resultado** del árbol.
- El resultado es una **forma sentencial** de G.

Propiedad: α es una forma sentencial de un árbol sii $S \Rightarrow^* \alpha$

Ejercicio

$$G = (\quad \{E, E_p, V, I, D\},$$
$$\{+, -, *, /, (,), x, y, z, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\},$$
$$P, E)$$

donde P tiene las siguientes producciones:

$$\begin{aligned}
E &\rightarrow E + E \mid E - E \mid E * E \mid E/E \mid E_p \\
E_p &\rightarrow V \mid I \mid (E) \\
V &\rightarrow x \mid y \mid z \\
I &\rightarrow DI \mid D \\
D &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}$$

Dibujar el árbol de parseo de las siguientes expresiones

- $1 + x$
- $69 * x - z$
- $(99 + 4 * y * (x - 678))$
- -1

Gramáticas Ambiguas

- Una GLC G es ambigua si alguna palabra en $L(G)$ tiene más de un árbol de parseo.
- Un LLC para el que toda GLC es ambigua es inherentemente ambiguo.
- La mayoría de los LLCs no son inherentemente ambiguos.
- Una GLC G ambigua, usualmente puede ser transformada a una equivalente G' que no es ambigua.
- En general, no existe un algoritmo finito para saber si una GLC es ambigüedad o no (no es decidible).

Gramáticas Ambiguas: dangling else

La siguiente gramática posee el problema del conocido como **dangling else**.

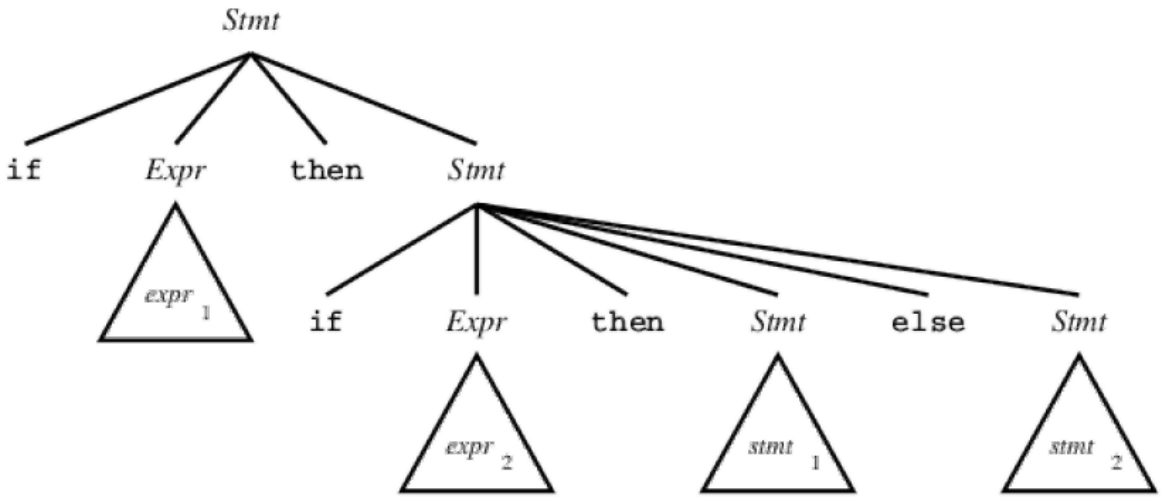
$$
\begin{aligned}
\text{Stmt} \rightarrow\ & \textbf{if}\ \text{Expr}\ \textbf{then}\ \text{Stmt} \\
\mid\ & \textbf{if}\ \text{Expr}\ \textbf{then}\ \text{Stmt}\ \textbf{else}\ \text{Stmt} \\
\mid\ & \mathit{other}
\end{aligned}
$$

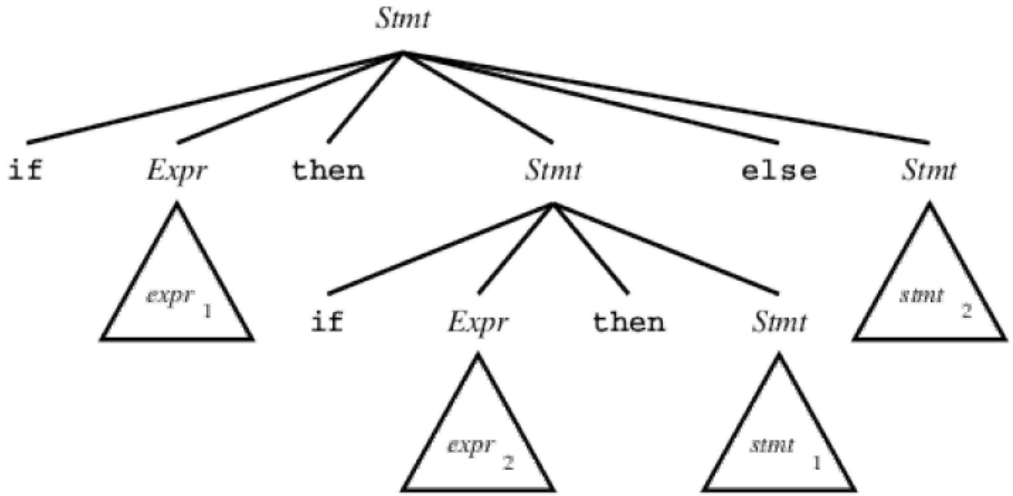
consideremos el programa

$$\textbf{if}\ expr_1\ \textbf{then if}\ expr_2\ \textbf{then}\ stmt_1\ \textbf{else}\ stmt_2$$

tiene 2 árboles de parseo posibles por lo que la **gramática es ambigua**

Gramáticas Ambiguas: dangling else





Desambiguar gramática

- Cada árbol de parseo tiene un significado distinto.
- Si $expr_1$ es $true$ y $expr_2$ es $false$, ¿Qué pasa en cada caso?
- La interpretación que usamos es:

Asociar cada **else** *al* **then** *más cercano que no este asociado a otro* **else**

¿Qué árbol preferimos?

- Tenemos que transformar la gramática en una equivalente pero sin ambigüedades.

Desambiguar gramática

Una sentencia entre un **then** y un **else** deben ser una sentencia **matcheada**

$$
\begin{aligned}
Stmt &\rightarrow MatchedStmt \\
&\mid UnmatchedStmt \\
MatchedStmt &\rightarrow \textbf{if}\ \text{Expr}\ \textbf{then}\ MatchedStmt \\
&\qquad\qquad \textbf{else}\ MatchedStmt \\
&\mid other \\
UnmatchedStmt &\rightarrow \textbf{if}\ \text{Expr}\ \textbf{then}\ Stmt \\
&\mid \textbf{if}\ \text{Expr}\ \textbf{then}\ MatchedStmt \\
&\qquad\qquad \textbf{else}\ UnmatchedStmt
\end{aligned}
$$

Ejercicio

Considera la gramática para expresiones aritméticas vistas anteriormente: ¿Es ambigua? Justifica tu respuesta.

- Las expresiones aritméticas se desambiguan mediante convenciones como precedencia de operadores y asociatividad.
- Las mismas ideas se pueden usar para desambiguar gramáticas en general.

Sintaxis Concreta y Abstracta

- Los árboles de parseo refieren a **sintaxis concreta**
  - Son cadenas sin estructuras.
  - Contienen poco detalles.
  - Su estructura no es única si la gramática es ambigua.
- Es conveniente trabajar con árboles de **sintaxis abstracta**
  - La sintaxis abstracta ya es un árbol por lo que no puede haber problemas de ambigüedad.
  - Las gramáticas se simplifican ya que no es necesario los formatos o paréntesis.

Sintaxis Concreta

La sintaxis concreta de un lenguaje de expresiones aritméticas simple es:

$$
\begin{array}{lll}
< digit > & ::= & \text{'0'} \mid \text{'1'} \mid ... \mid \text{'9'} \mid \\
< nat > & ::= & < digit > \mid < digit > < nat > \\
< intexpr > & ::= & < nat > \\
 & & \mid < intexpr > \text{'+'} < intexp > \\
 & & \mid < intexpr > \text{'*'} < intexp > \\
 & & \mid \text{'('} < intexpr > \text{')'}
\end{array}
$$

La sintaxis es ambigua. Podemos desambiguarla especificando asociatividad y precedencia de operadores.

Sintaxis Abstracta

Nos interesan los naturales y expresiones como entidades abstractas.
El árbol de sintaxis abstracta (AST) es:

$$\begin{aligned} < intexp > ::= & < nat > \\ & | < intexpr > + < intexp > \\ & | < intexpr > * < intexp > \end{aligned}$$

Los AST no tienen ambigüedades.



Definimos Términos

Los AST me definen un conjunto de términos.

El conjunto de términos del lenguaje es el menor conjunto T tal que:

1. $n \in \mathbb{N}$ entonces $n \in T$
2. $t, u \in T$ entonces $t + u \in T$
3. $t, u \in T$ entonces $t * u \in T$

De igual forma podemos definir el conjunto mediante **reglas de inferencia**

$$\frac{n \in \mathbb{N}}{n \in T} \qquad \frac{t \in T \quad u \in T}{t + u \in T} \qquad \frac{t \in T \quad u \in T}{(t * u) \in T}$$

Los símbolos $+$ y $*$ son arbitrarios y pueden ser diferentes a sintaxis concreta.

Implementación

En Haskell podemos usar un tipo de datos algebraico para representar el AST de un lenguaje:

```
data IntExp = Num Int
            | Sum IntExp IntExp
            | Prod IntExp IntExp
```

Los elementos de IntExp son árboles con etiquetas Num, Sum y Prod en sus nodos.
En la implementación se ve bien claro que tratamos con árboles y que los nombre de los constructores son arbitrarios.

Lenguaje de Expresiones Aritméticas

Consideremos el AST de un lenguaje:

$$
\begin{aligned}
t ::= & \; true \\
| & \; false \\
| & \; \textbf{if } t \textbf{ then } t \textbf{ else } t \\
| & \; 0 \\
| & \; succ \; t \\
| & \; pred \; t \\
| & \; iszero \; t
\end{aligned}
$$

Un programa es un término del lenguaje:

- **if** $false$ **then** 0 **else** 1 evalúa a 1 ?
- $iszero \; (pred \; (succ \; 0))$ evalúa a $true$?
- $succ \; true$ evalúa a ... ?

$$1 \equiv (succ \; 0)$$

Notación

- Las metavariables están en el metalenguaje. Se usan para representar entidades del lenguaje objeto.
- La metavariable t, letras cercanas como u, s y r , y variaciones como t_1 o t_0 representan términos del lenguaje.
- Un término es una frase que computa.
- Una expresión es más general (por ejemplo podríamos tener una expresión de tipos).
- Salvo que se aclare siempre nos referimos a la sintaxis abstracta.
- Los paréntesis se usan exclusivamente para expresar un árbol como una cadena de texto.

Términos en forma Inductiva

Podemos definir el conjunto de términos inductivamente:

Los términos son el menor conjunto $\mathcal{T}$ tal que:

1. $\{true, false, 0\} \subseteq \mathcal{T}$
2. si $t_1 \in \mathcal{T}$ entonces $\{succ\ t_1, pred\ t_1, iszero\ t_1\} \subseteq \mathcal{T}$
3. si $t_1, t_2, t_3 \in \mathcal{T}$ entonces **if** t_1 **then** t_2 **else** $t_3 \in \mathcal{T}$

El requerimiento que $\mathcal{T}$ sea el conjunto más chico dice que en $\mathcal{T}$ sólo están los elementos requeridos por las tres cláusulas.

Recordar que $\mathcal{T}$ es un conjunto de árboles.

Términos por Reglas de Inferencia

$$true \in \mathcal{T} \qquad false \in \mathcal{T} \qquad 0 \in \mathcal{T}$$

$$\frac{t_1 \in \mathcal{T}}{succ\ t_1 \in \mathcal{T}} \qquad \frac{t_1 \in \mathcal{T}}{pred\ t_1 \in \mathcal{T}} \qquad \frac{t_1 \in \mathcal{T}}{iszero\ t_1 \in \mathcal{T}}$$

$$\frac{t_1 \in \mathcal{T} \quad t_2 \in \mathcal{T} \quad t_3 \in \mathcal{T}}{\textbf{if } t_1 \textbf{ then } t_2 \textbf{ else } t_3 \in \mathcal{T}}$$

En este estilo, usualmente se deja implícito que uno está interesado en el menor conjunto que satisface las reglas.
A menudo decimos "reglas de inferencia", pero cuando contienen metavariables, son en realidad **esquemas de reglas**.

Términos en forma concreta

Para cada $i \in \mathbb{N}$ definimos un conjunto S_i

$$
\begin{aligned}
S_0 &= \emptyset \\
S_{i+1} &= \{true, false, 0\} \\
&\cup \{succ\ t_1, pred\ t_1, iszero\ t_1 \mid t_1 \in S_i\} \\
&\cup \{\textbf{if}\ t_1\ \textbf{then}\ t_2\ \textbf{else}\ t_3 \mid t_1, t_2, t_3 \in S_i\}
\end{aligned}
$$

Luego S queda definido como:

$$S = \bigcup_{i \in \mathbb{N}} S_i$$

Ejercicio: ¿Cuántos elementos tiene S_3?
Solución: $|S_{i+1}| = |S_i| \uparrow 3 + |S_i| * 3 + 3$, $|S_0| = 0$

$$S_3 = 59439$$

Definiciones Equivalentes

La definición **inductiva** y la **dada por reglas** caracterizan los términos como un conjunto que satisface ciertas propiedades.
La definición **concreta** muestra como construir el conjunto como el límite de una secuencia.

$$\mathcal{T} \equiv S$$

El conjunto $\mathcal{T}$ fue definido como el menor conjunto que satisface ciertas condiciones. Para probar el teorema basta:

1. probar que S satisface las condiciones.
2. probar que todo conjunto que satisface las condiciones contiene a S

Resumen

- Los lenguajes de programación son lenguajes formales.
- En general, basta gramáticas libres de contexto para describir su sintaxis concreta.
- La sintaxis concreta se refiere a cadenas, por lo que puede ser ambigua.
- La sintaxis abstracta trata con árboles, por lo que no es ambigua.
- Para procesar un programa, el primer paso es obtener su árbol de sintaxis abstracta (AST).
- Un AST es un conjunto de términos definidos inductivamente.
- Caracterizaciones. Pruebas por inducción estructural.

Referencias

- Types and Programming Languages. B.C. Pierce.
- Foundations of Programming Languages. J.C. Mitchell
- Transparencias de clase. Nociones Básicas (ALP). Phd Mauro Jaskelioff.